

Introduction to Containers with Podman





Objectives

- Building and managing containers with Podman for deployment on a Kubernetes and OpenShift 4 cluster.
- This course prepares the student to take the Red Hat Certified Specialist in containerized applications exam (EX188).

Audience

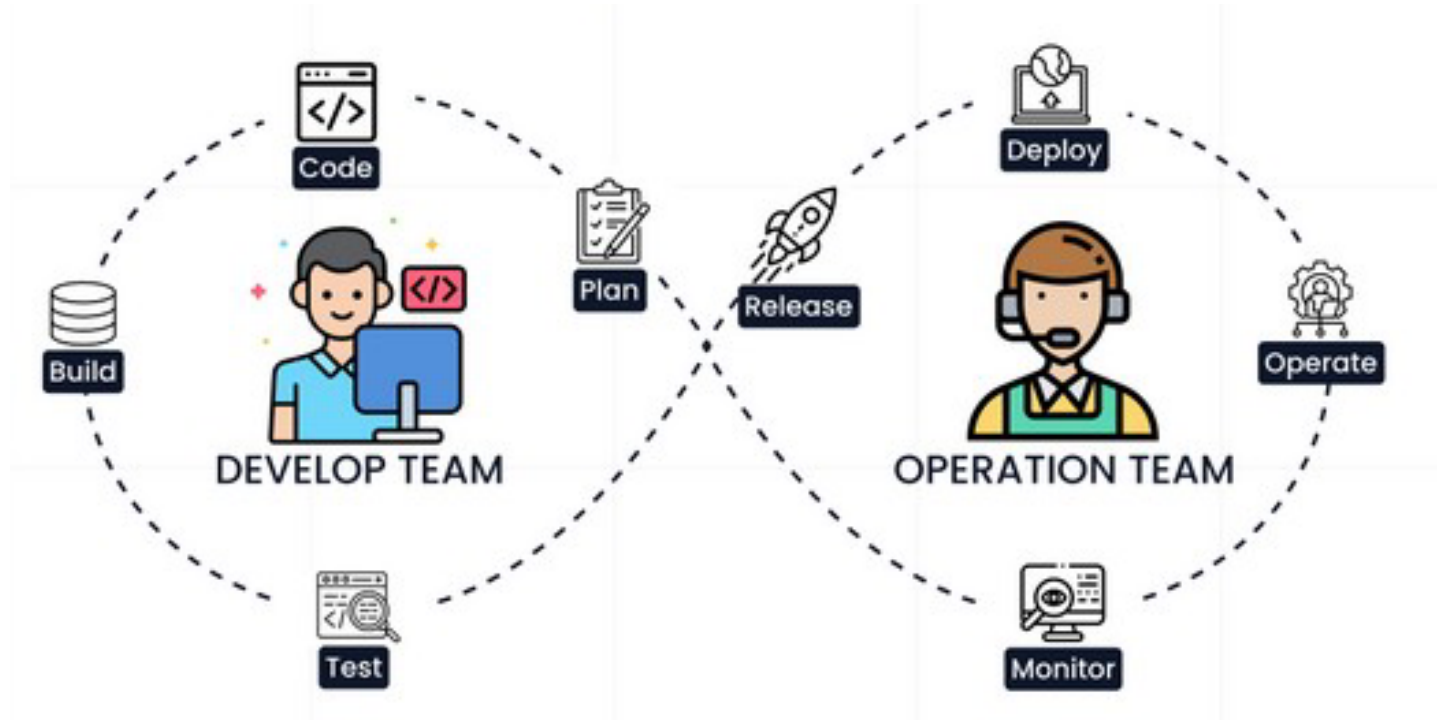
- Developers, Devops and site reliability engineers that are new to container technology.



Who is a DevOps engineer?

- A DevOps engineer is an IT professional with knowledge of **development** and **operations**.
 - The development part involves writing codes and scripts,
 - The operations part includes managing on-premises and/or cloud infrastructure and system infrastructure.
- In traditional software development, teams often work in silos, meaning they operate separately with limited collaboration. This makes communication difficult, and each team may have different priorities and timelines that don't align well with others.
- DevOps helps to **bridge the gap between development teams and operations teams**.
- DevOps experts often work in a collaborative surrounding where they can collaborate with software engineers and IT teams.

DevOps Process





Environment Setup

- **On Windows**

You can use one of the following options:

- WSL (Windows Subsystem for Linux). It provides a Linux environment inside Windows and works well with container tools.
- Minikube – Useful for running a local Kubernetes cluster.
- Podman – A container engine similar to Docker, supported on Windows.
- Cygwin – Provides a Linux-like environment.

- **On Linux:**

Supported distributions include:

- Ubuntu
- CentOS
- Red Hat Enterprise Linux (RHEL) → **Recommended and will be used in the exam.**

Chapter 1. Introduction and Overview of Containers



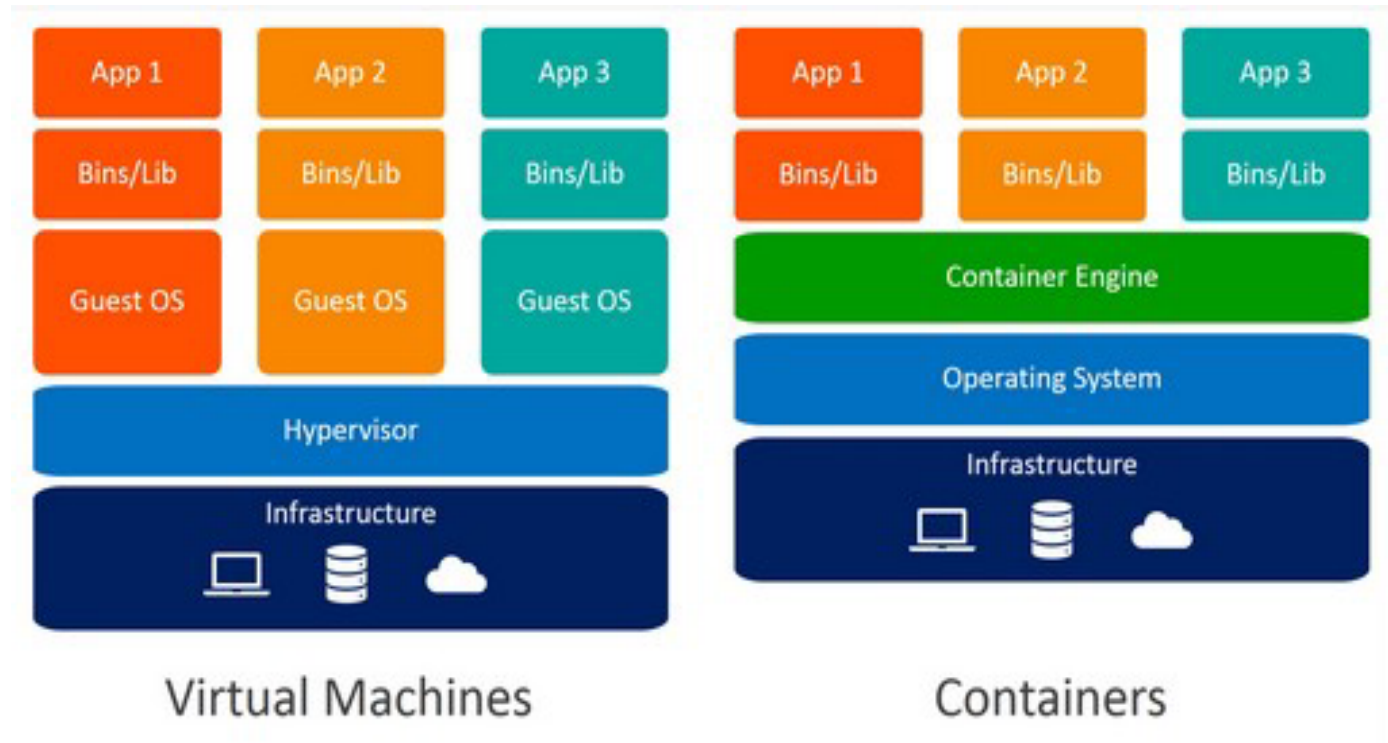
Outline



- What is virtualization and Containerization.
- Virtualization VS Containerization.
- Why we use Containers?
- Advantage and Disadvantage of Containerization.
- What is Kubernetes and its features.
- What is RHOCp and its features .



Virtualization vs Containerization





Virtual machines

- A **virtual machine (VM)** is a virtual environment that **simulates a physical computer** in software form.
- **Virtualization** means **partitioning physical server** into multiple virtual servers.
- A **hypervisor** is the **software** layer that coordinates Vms.
 - It serves as an **interface** between the **VM** and the underlying **physical hardware**, ensuring that each has access to the physical resources it needs to execute.
 - Example: VirtualBox, VMware Workstation, Hyper-V.
- **WHEN to use?** A **VM is useful when** an additional full computing environment is required, such as when an application requires specific, dedicated hardware.
- Additionally, a **VM is preferable when** an application **requires a non-Linux operating system or a different kernel from the host**.
- With VMs, interoperability is uncommon. A VM that runs on one hypervisor is usually not guaranteed to run on a different hypervisor. In contrast, containers that follow the OCI specification do not require a particular container engine to function. Many container engines can function as drop-in replacements for each other.



Containers

- **Container** is an encapsulated process that includes the required runtime dependencies for the program to run.
- In a container, application-specific libraries are independent of the **host operating system libraries**.
- Libraries and functions that are not specific to the containerized application are provided by the **operating system and kernel**.
- The provided libraries and functions help to ensure that the container remains compact, and that it can quickly execute and stop as needed.
- A container engine creates a union file system by merging container image layers. Because **container image layers are immutable**, a container engine adds a writable layer for runtime file modifications. Containers are **ephemeral by default**, which means that the container engine removes the writable layer when you remove the container.



Containers

- Containers use **Linux kernel features** such as:
 - 1) **Namespaces**
 - It provides the functionality to **isolate processes within containers** from each other and from the host system.
 - 2) **Control Groups (cgroups).**
 - For example, containers use cgroups for resource management, such as CPU time allocation and system memory.
- As such, the environment within a container is Linux-based, regardless of the host operating system. When using containers on non-Linux operating systems, these Linux-specific features are often virtualized by the container engine implementation.
- Containerization originated from technologies such as chroot (change root), a method to partially or fully isolate an environment, and evolved to the **Open Container Initiative (OCI)**.
- **OCI** is a governance organization that defines standards for creating and running containers.
- Most container engines conform to the OCI specifications, so developers can confidently build their deployable target artifacts to run as OCI containers.



Virtual Machines versus Containers

Key Factor	Virtualization	Containerization
size	GB (Heavy Weight)	MB (Light Weight)
Resource Management	Each VM has its own OS	All Containers share the same OS
Where Virtualization Occurs	HW Virtualization	OS Virtualization
Startup time	Minute	Second
Isolation	More secure and fully isolated.	Less secure and isolated at the process level.



Virtual Machines versus Containers

Key Factor	Virtualization	Containerization
Portability	Limited portability across environments	Write Once, Run Anywhere
Scaling issues	Scaling a monolithic app meant starting a new VM, which could take minutes and wasted resources since the entire OS had to boot.	Faster scaling & deployment: Tools like Docker and orchestrators like Kubernetes allow containers to be deployed, replicated, and rolled back
	Environment mismatch: “It works on my machine” was a common problem, because apps behaved differently across development, testing, and production environments.	Microservices-friendly: Instead of one large monolithic app, systems can be split into smaller services, each in its own container, making scaling and updates easier.



Virtual Machines versus Containers

Key Factor	Virtualization	Containerization
	Slow delivery: Updating even a small feature meant redeploying the entire application or spinning up more VMs, making releases slow and risky.	Modern CI/CD pipelines: Containers integrate seamlessly with automated pipelines, ensuring smoother collaboration between developers, testers, and operations teams.



Advantages of Containerization

- Containerization provides many advantages for the development process, **such as easier testing and deployments, by providing tools for stability, security, and flexibility.**
- **Scalability:** One of the greatest advantages of containers for developers is the ability to scale.
- **Stability:** container images are a **stable target** for developers. Since libraries are included in the container image, developers avoid dependency issues and OS requirements. This integration removes variability between testing and production environments. For example, a container with a specific version of Python ensures the same version is used in every environment.
- **Resource Efficiency:** Because containers share the kernel of the host operating system, running a separate operating system for each instance is no longer necessary. As a result, **resource efficiency is greatly improved.**
- A multi-container application is distributed across several containers. You can run the containers from the same image for high availability (HA) replicas, or from several different images.
 - For example, a developer can create an application that includes a database container that runs separately from the application's web API container. An application can rely on the container management software to provide HA replicas with multi-container options, such as Podman Pods or the compose-spec specification.



Disadvantages of Containerization

- **Limited Isolation:** Containers share the underlying host kernel, which introduces a degree of vulnerability to the host system. One of the biggest containerization disadvantages is if one container's security is breached, it affects the security of all other containers, and not just the host itself.
- **Complexity of Orchestration:** Managing containerized applications at scale can be challenging, requiring the implementation of container orchestration tools, like Kubernetes, for automated deployment, scaling, and service discovery.



Kubernetes “K8s”

- Kubernetes is an **orchestration service** that simplifies the deployment, management, and scaling of containerized applications.
- It manages complex pools of resources, such as CPU, RAM, storage, and networking.
- Kubernetes provides high uptime and fault tolerance for containerized application deployments, removing the concern that developers might have regarding how their applications use resources.
- Kubernetes clusters provide a modern **container platform** that addresses the concerns and challenges of running applications at scale.



Features of Kubernetes

- **Horizontal scaling**

Applications can scale up and down manually or automatically with a configuration set, with either the Kubernetes command-line interface or the web UI.

- **Self-healing**

Kubernetes can use user-defined health checks to monitor pods to restart and reschedule them in the event of failure.

- **Automated rollout**

Kubernetes can gradually roll out updates to your application's containers while checking their status. If something goes wrong during the rollout, then Kubernetes can roll back to the previous version of the deployment.

- **Secrets and configuration management**

You can manage the configuration settings and secrets of your applications without rebuilding containers. Application secrets can be usernames, passwords, and service endpoints, or any configuration setting that must be kept private.



Features of Kubernetes

- **Service discovery and load balancing**
 - Kubernetes enables inter-service communication by assigning a single DNS entry to each set of containers.
 - To enable the cluster to change the container's location and IP address, the requesting service must know the target's DNS name.
 - As a result, Kubernetes **can load-balance the request across the pool of containers that provide the service.**
 - For example, Kubernetes can evenly split incoming requests to an NGINX web server by **taking into account the availability of the NGINX pods.**



Red Hat OpenShift Container Platform Overview

- Red Hat OpenShift Container Platform (RHOCP) is a set of modular components and services that are **built on top of the Kubernetes container infrastructure**.
- RHOCP adds capabilities to a production platform, such as remote management, multitenancy (same OpenShift cluster can be shared by multiple teams or projects), increased security, monitoring and auditing, application lifecycle management, and self-service interfaces for developers.



Red Hat OpenShift Container Platform Features

- **Developer workflow**

Integrates a built-in container registry, **Continuous Integration/Continuous Delivery (CI/CD) pipelines**, and **Source-to-Image (S2I)**, a tool to build artifacts from source repositories to container images.

- **Routes**

Exposes services to the outside world easily.

- **Metrics and logging**

Includes a built-in and self-analyzing metrics service and aggregated logging.

- **Unified UI**

Provides unified tools and an integrated UI to manage the different capabilities.

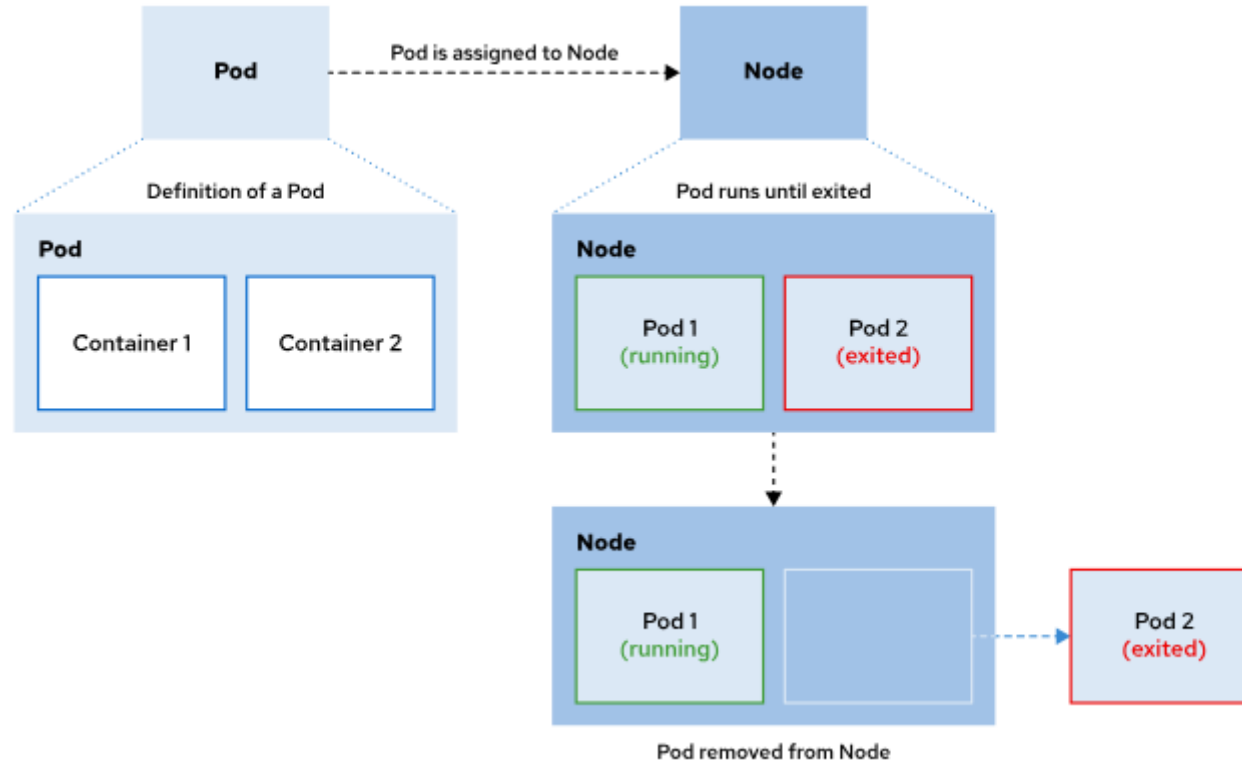
Lifecycle of Applications in Red Hat OpenShift Container Platform



The following figure illustrates the basic lifecycle of an application that is deployed in a RHOC P cluster:

- Starts with the definition of a pod and the containers that it is composed of, which contain the application.
- Pods are assigned to a healthy node.
- Pods run until their containers exit.
- Pods and their containers are removed from the node.

Lifecycle of Applications in Red Hat OpenShift Container Platform



QUIZ 1.2 & 1.4

